

# RecLM: RECOMMENDATION INSTRUCTION TUNING

Yangqin Jiang<sup>1</sup>, Yuhao Yang<sup>1</sup>, Lianghao Xia<sup>1</sup>, Da Luo<sup>2</sup>, Kangyi Lin<sup>2</sup>, Chao Huang<sup>1\*</sup>

University of Hong Kong<sup>1</sup>, Tencent<sup>2</sup>

{mrjiangyq99, chaohuang75}@gmail.com yuhao-yang@outlook.com aka\_xia@foxmail.com

## ABSTRACT

Modern recommender systems aim to deeply understand users’ complex preferences through their past interactions. While deep collaborative filtering approaches using Graph Neural Networks (GNNs) excel at capturing user-item relationships, their effectiveness is limited when handling sparse data or zero-shot scenarios, primarily due to constraints in ID-based embedding functions. To address these challenges, we propose a model-agnostic recommendation instruction-tuning paradigm that seamlessly integrates large language models with collaborative filtering. Our proposed Recommendation Language Model (RecLM) enhances the capture of user preference diversity through a carefully designed reinforcement learning reward function that facilitates self-augmentation of language models. Comprehensive evaluations demonstrate significant advantages of our approach across various settings, and its plug-and-play compatibility with state-of-the-art recommender systems results in notable performance enhancements. The implementation of our RecLM framework is publicly available at: <https://github.com/HKUDS/RecLM>.

## 1 INTRODUCTION

Recommendation systems serve as essential components of modern web applications, helping users navigate through the vast digital information landscape. These systems provide personalized suggestions across diverse platforms, including product recommendations on e-commerce platforms (Wang et al., 2020; Wu et al., 2018), content discovery on social networking sites (Jamali & Ester, 2010; Zhang et al., 2021), and viewer-tailored suggestions on video sharing platforms (Zhan et al., 2022; Jiang et al., 2024). At the core of these systems lies Collaborative Filtering (CF), a widely adopted approach that harnesses collective user preferences to generate personalized recommendations.

Current recommender systems predominantly operate within the user/item ID paradigm, where training data consists primarily of mapped user and item indices. While this approach has driven significant advancements in recommendation technology, particularly in data-rich scenarios (Yao et al., 2021; Yuan et al., 2023), it faces several fundamental limitations. Most notably, these systems encounter substantial challenges in cold-start scenarios and struggle to generalize effectively in zero-shot learning situations. The inherent dependency on ID-based representations becomes particularly problematic in completely cold-start settings, where systems fail to generate meaningful representations for new items, ultimately compromising their ability to deliver accurate recommendations.

Addressing the fundamental cold-start challenge in ID-based recommendation systems requires moving beyond traditional ID-based embeddings to leverage external features (*e.g.*, textual or visual information) for generating user and item representations. However, this promising approach encounters significant practical challenges in real-world applications, primarily stemming from two critical issues: data incompleteness and quality concerns. On the incompleteness front, users frequently withhold personal information due to privacy concerns, leading to substantial gaps in external features. Simultaneously, existing data often suffers from quality issues, manifesting as misleading tags, irrelevant product specifications, or unreliable item descriptions. These challenges collectively underscore the critical need for robust methods to extract accurate, relevant, and high-quality features from inherently noisy and incomplete side information—a prerequisite for achieving effective recommendation generalization in data-scarce scenarios.

\*Chao Huang is the corresponding author.

---

**Contribution.** The exceptional generalization and reasoning capabilities of Large Language Models (LLMs) motivate a novel solution to cold-start recommendation challenges through specialized profiling systems based on external side information. This approach centers on developing effective language models customized for recommendation tasks, addressing two critical challenges: (i) designing robust mechanisms for LLMs to generate accurate profile representations that capture essential recommendation characteristics, especially for users or items lacking historical interactions, and (ii) developing techniques for LLMs to distill high-quality profiles from noisy feature sets while preserving the integrity of user-item interaction patterns and behavioral context.

To address these challenges, we propose RecLM, a novel recommendation instruction tuning paradigm that revolutionizes how LLMs understand behavioral contexts in recommender systems. While LLMs demonstrate superior natural language processing capabilities, they fundamentally lack the ability to model complex user-item interactions and behavioral preferences. Our approach tackles this limitation through two key technical innovations: (1) a seamless integration mechanism that fuses external user-item features with collaborative interaction patterns through specialized instruction tuning, enabling effective cold-start profiling without direct supervision signals, and (2) a reinforcement learning-based personalized feature enhancement framework that systematically reduces noise and mitigates over-smoothing effects inherent in collaborative filtering. This model-agnostic framework can be plugged into existing recommender systems to significantly enhance their generalization capacity, particularly in data-scarce scenarios. Our main contributions are as follows:

- **Model-Agnostic Framework.** We introduce a model-agnostic instruction tuning framework RecLM. It can be seamlessly integrated into existing recommender systems as a plug-and-play component, significantly enhancing their generalization capacity in scenarios with data scarcity.
- **Enhancing Profiling System.** We seamlessly integrate large language models with collaborative filtering to enhance user profiling and representation, particularly in cold-start scenarios, where current methods often struggle. Additionally, our approach employs reinforcement learning to refine profile quality, effectively addressing challenges associated with data noise and over-smoothing.
- **Comprehensive Evaluation.** We integrate RecLM with a range of state-of-the-art recommenders to assess the effectiveness of our approach across various settings. This includes conducting ablation studies and efficiency evaluations. Additionally, we carry out extensive experiments in real-world industrial recommendation scenarios, demonstrating the practicality and scalability of RecLM.

## 2 METHODOLOGY

### 2.1 ID-BASED COLLABORATIVE FILTERING

In the ID-based collaborative filtering (CF) paradigm, the primary goal is to optimize the ID embeddings of users and items. This optimization aims to accurately capture and represent user preferences for items, while considering the interaction patterns of users and items that are similar. Formally, we have a set of users denoted as  $\mathcal{U} = \{u_1, \dots, u_I\}$ , and a set of items denoted as  $\mathcal{V} = \{v_1, \dots, v_J\}$ . Each user and item is assigned initial ID embeddings, represented as  $\mathbf{x}_u$  and  $\mathbf{x}_v \in \mathbb{R}^d$  respectively. The objective is to obtain optimized user and item representations, denoted as  $\mathbf{e}_u, \mathbf{e}_v \in \mathbb{R}^d$ , through a recommender model  $\mathcal{R}(\mathbf{x}_u, \mathbf{x}_v)$ . This model aims to maximize the posterior distribution  $p(\mathbf{e}|\mathcal{X}) \propto p(\mathcal{X}|\mathbf{e})p(\mathbf{e})$ . The predicted likelihood of user-item interaction, denoted as  $\hat{y}_{u,v}$ , is derived by performing a dot product between the user and item representations, as follows:  $\hat{y}_{u,v} = \mathbf{e}_u^\top \cdot \mathbf{e}_v$ .

While state-of-the-art recommender systems operating within the ID-based collaborative filtering paradigm have exhibited remarkable performance, they face significant challenges when tasked with handling cold-start recommendation scenarios, especially in situations where data scarcity is prevalent. The primary obstacle that ID-based recommenders encounter in these situations stems from the lack of past interaction history for newly introduced items. This absence of accumulated user engagement data disrupts the optimization paradigm that these systems typically rely upon, thereby making it considerably more difficult to generate accurate and meaningful representations for these items. Consequently, ID-based recommenders may encounter considerable difficulties in effectively modeling and understanding the inherent characteristics and preferences associated with these cold-start items, leading to a notable decline in the overall performance, particularly in zero-shot scenarios where no prior interaction data is available for certain items.

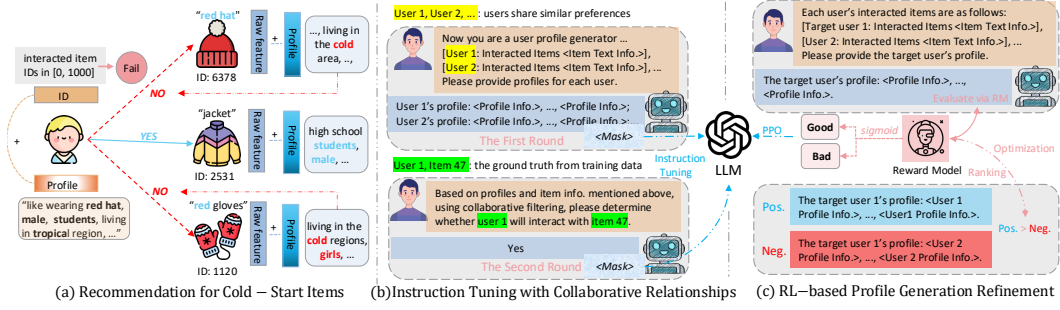


Figure 1: The Overall Framework of the Proposed RecLM.

## 2.2 TEXT-EMPOWERED USER/ITEM REPRESENTATIONS

To address the challenge of cold-start items in zero-shot recommendation scenarios, we propose a novel approach that leverages textual side features for generalized and adaptable user and item representation learning. Specifically, we seek to replace the traditional ID-based embeddings with the side information associated with the items, namely their text descriptions, represented as  $\mathbf{F} \in \mathbb{R}^{|\mathcal{V}| \times d_t}$ . To accomplish this, we utilize a project layer with a multi-layer perceptron  $T_{raw}$  to map the raw textual features  $\mathbf{f} \in \mathbb{R}^{d_t}$  into a lower-dimensional latent space  $\mathbb{R}^d$ . The resulting representation  $\hat{\mathbf{f}} \in \mathbb{R}^d$  is then used as the initial item representation in our model.

$$\hat{\mathbf{f}}_v = T_{raw}(\mathbf{f}). \quad (1)$$

This text-driven approach enables a seamless fusion of collaborative signals with textual semantics, empowering our recommender to capture user preferences with unparalleled accuracy. Using rich textual features as initial item representations, we optimize user ID embeddings based on observed interactions. This injects valuable collaborative cues into the user and item representations, making them adaptable to evolving preferences. This synergistic learning equips our model to deeply comprehend users' affinities for text-based items. Crucially, it also endows the system with zero-shot prediction capabilities for cold-start items lacking prior interaction data.

**LLM-enhanced User/Item Profiling** To further empower user representations with the rich textual semantics provided by large language models (LLMs), we propose generating user profile information that can reflect their interaction preferences. Specifically, item profiles can be derived from the profiles of users who frequently interact with them. This approach proves valuable in capturing user preferences and facilitating accurate recommendations for cold-start items.

On the user side, the ID embedding  $\mathbf{x}_u \in \mathbb{R}^d$  integrates with the user profile  $\mathbf{p}_u \in \mathbb{R}^{d_t}$ , allowing the system to leverage the user's ID-based embedding and their generated profile, which can capture more nuanced preferences. On the item side, the raw text  $\mathbf{x}_v$  combines with the item profile  $\mathbf{p}_v \in \mathbb{R}^{d_t}$ , enabling the system to understand the item's characteristics and how they align with user preferences.

$$\hat{\mathbf{f}}_u^{aug} = \Psi(\mathbf{x}_u | T_{pro}(\mathbf{p}_u)), \quad \hat{\mathbf{f}}_v^{aug} = \Psi(\hat{\mathbf{f}}_v | T_{pro}(\mathbf{p}_v)). \quad (2)$$

Our framework employs a dual-MLP architecture for effective multi-modal fusion: an initial MLP encoder  $\Psi(\cdot)$  consolidates heterogeneous features, followed by a profile transformation network  $T_{pro}$  that projects the unified embeddings into the model's latent space. This hierarchical process generates enriched user and item representations  $\hat{\mathbf{f}}_u^{aug} \in \mathbb{R}^d$  and  $\hat{\mathbf{f}}_v^{aug} \in \mathbb{R}^d$ , capturing comprehensive behavioral patterns. To further enhance representation quality, we leverage state-of-the-art LLMs as profile augmentation engines, generating supplementary semantic information that significantly boosts our recommender system's modeling capacity.

## 2.3 INSTRUCTION TUNING WITH COLLABORATIVE RELATIONSHIPS

Our RecLM framework seeks to align the collaborative relationships with the textual semantic representation space through an innovative recommendation instruction tuning paradigm that seamlessly integrates user collaborative relationships into the LLM-based profiling process. The framework

employs a two-phase approach: first, it utilizes knowledge distillation and dialogue-based instruction tuning to preserve high-order collaborative similarities between users and items, generating high-quality and informative user profiles. Subsequently, it leverages these refined user profiles to generate semantically aligned item profiles, resulting in a cohesive representation space that effectively captures both individual characteristics and collaborative patterns.

### 2.3.1 LLM FINE-TUNING WITH COLLABORATIVE SIGNALS

Our approach leverages llama2-7b-chat as the base model, augmented with a novel collaborative instruction tuning framework that captures both user-item interactions and profile generation capabilities. Specifically, we design specialized prompt templates incorporating collaborative signals, and construct input-output pairs using ChatGPT-3.5 to capture diverse user-item relationship patterns. Here, the ChatGPT-3.5 is utilized as a powerful LLM with strong text summary capacity to generate text profile to characterize the interaction patterns among users and items, and the collaborative relationships among them, and result in textual instructions used for collaborative instruction tuning.

### 2.3.2 COLLABORATIVE INSTRUCTION TUNING

In our recommendation instruction tuning paradigm, we propose to inject higher-order collaborative relational context through a novel two-turn collaborative instruction tuning paradigm. By incorporating collaborative signals that capture complex user-user and user-item relationships, this approach transcends the limitations of methods that rely solely on direct user-item interactions. Our framework serves a dual purpose: it empowers LLMs to generate richer and more nuanced user profiles by leveraging the collaborative network dynamics among users and items, while simultaneously addressing the fundamental challenge of insufficient direct supervision in profile generation tasks.

**Profile Generation with Two-turn Dialogue.** A fundamental challenge in LLM-based profile generation lies in the absence of ground truth data for direct evaluation, forcing practitioners to rely on indirect assessment through downstream recommendation performance. To address this limitation, we propose a novel two-turn collaborative instruction tuning paradigm that provides explicit supervision signals. Unlike conventional approaches that struggle with profile quality assessment, our framework leverages collaborative user relationships to guide LLMs in generating high-quality profiles through structured dialogue interactions.

• **First Turn - Collaborative Profile Generation:** The initial turn takes input query  $\mathcal{Q}$  containing both the target user’s historical item interactions and those of similar users identified from their collaborative neighborhood. Here, user similarities are measured by the encoded user embeddings from LightGCN model (He et al., 2020). The LLM generates output response  $\mathcal{R}$  consisting of user profiles that capture collaborative patterns.

$$\mathcal{Q}_{fir.} = Prompt(u_t, \{u_n\}, \mathcal{V}_t, \{\mathcal{V}_n\}), \quad \mathcal{R}_{fir.} = Prompt(u_t, \{u_n\}, \mathcal{P}_t, \{\mathcal{P}_n\}). \quad (3)$$

• **Second Turn - Supervised Interaction Prediction:** The second turn reformulates the instruction-tuning task as an interaction prediction problem, aiming to enhance profile generation by incorporating explicit user-item preference signals. Specifically, the input query  $\mathcal{Q}$  prompts whether target user  $u_t$  will interact with candidate item  $v_t$ , while the output response  $\mathcal{R}$  corresponds to the ground truth interaction status (*i.e.*, *yes* or *no*) from the training dataset. To ensure balanced and informative training signals, we employ a systematic sampling strategy: For positive samples (50%), we select  $v^+$  from the user’s interaction history  $\mathcal{V}_t$  under the constraint that  $v^+$  also appears in similar users’ history  $\mathcal{V}_n$ , while removing it from  $\mathcal{V}_t$  in first-turn instructions to prevent information leakage. For negative samples (50%), we select  $v^-$  from similar users’ history  $\mathcal{V}_n$  while ensuring no historical interaction exists between  $u_t$  and  $v^-$  in the training data. This balanced sampling approach maintains unbiased training objectives while preserving crucial collaborative signals for effective profile generation.

$$\mathcal{Q}_{sec.} = \begin{cases} Prompt(u_t, v^+), & pos. \text{ samp.} \\ Prompt(u_t, v^-), & neg. \text{ samp.} \end{cases} \quad \mathcal{R}_{sec.} = \begin{cases} \text{Yes}, & pos. \text{ samp.} \\ \text{No}, & neg. \text{ samp.} \end{cases} \quad (4)$$

**Tuning Strategy.** For multi-turn dialogue instruction-tuning, our objective is to utilize LLM-generated responses  $\mathcal{R}$  for weight updates while excluding queries  $\mathcal{Q}$ . Traditional single-turn dialogue tuning,

when applied to our paradigm, considers  $\mathcal{Q}_{fir.}$ ,  $\mathcal{R}_{fir.}$ , and  $\mathcal{Q}_{sec.}$  as inputs, with only  $\mathcal{R}_{sec.}$  being predicted. This approach limits weight updates to losses from  $\mathcal{R}_{sec.}$  alone, failing to leverage the training data in multi-turn dialogues. Notably,  $\mathcal{R}_{fir.}$  contains rich textual information as multiple user profiles, which guides the generation of  $\mathcal{R}_{sec.}$  in subsequent turns. In contrast,  $\mathcal{R}_{sec.}$  consists of simpler binary responses (e.g., *yes* or *no*). Disregarding the valuable information in  $\mathcal{R}_{fir.}$  and relying on  $\mathcal{R}_{sec.}$  for fine-tuning would significantly limit the model’s learning capacity.

To address these limitations, we introduce a two-turn dialogue tuning approach. Specifically, we concatenate dialogues from both turns and apply masking techniques to differentiate between  $\mathcal{Q}$  and  $\mathcal{R}$  components. During training, the model computes losses exclusively from  $\mathcal{R}$ -marked segments, enabling both  $\mathcal{R}_{fir.}$  and  $\mathcal{R}_{sec.}$  to contribute to parameter optimization. This unified training strategy maximizes information utilization while effectively capturing collaborative relationships.

**Inference Prompt.** Following instruction-tuning, we design an inference prompt that integrates target user interactions  $\mathcal{V}_t$  with neighbor histories  $\mathcal{V}_n$ . This unified prompt guides LLMs to generate collaborative-aware user profiles by leveraging inter-user preference patterns.

$$\mathcal{Q}_{inf.} = \text{Prompt}(u_t, \{u_n\}, \mathcal{V}_t, \{\mathcal{V}_n\}). \quad (5)$$

## 2.4 REFINING PROFILE GENERATION THROUGH REINFORCEMENT LEARNING

We propose a reinforcement learning framework to enhance LLM-generated user profiles, focusing on improving accuracy and personalization. Our approach addresses two fundamental challenges: **i) Prompt-Training Discrepancy**—the inherent mismatch between inference (single-profile generation) and fine-tuning (multi-profile generation) phases that introduces potential inconsistencies in profile generation, and **ii) Personalization-Collaboration Trade-off**—while leveraging collaborative information enhances overall performance, it risks diluting individual user characteristics, analogous to the over-smoothing phenomenon observed in our collaborative instruction-tuning paradigm.

To address these challenges, we develop a reinforcement learning-based fine-tuning paradigm inspired by RLHF (Reinforcement Learning from Human Feedback)(Stiennon et al., 2020). Our approach consists of two key components: a reward model that evaluates LLM-generated profile quality, and an optimization procedure using Proximal Policy Optimization (PPO)(Schulman et al., 2017) that refines the LLM based on these reward signals. Through this iterative process, we progressively enhance the LLM’s ability to generate more accurate and personalized user profiles.

**Reward Model.** We design a reward model to evaluate the quality of LLM-generated outputs from a human perspective. Specifically, given an input pair  $[\mathcal{Q}, \mathcal{R}]$ , the model produces a scalar value indicating the response quality. The reward model is optimized using the loss function  $\mathcal{L}_{rm}$ :

$$\mathcal{L}_{rm} = - \sum_{i=0}^{\mathcal{I}} \mathbb{E}_{(\mathcal{Q}_i, \mathcal{R}_i^+, \mathcal{R}_i^-) \sim \mathcal{D}} [\log(\sigma(r_\theta(\mathcal{Q}_i, \mathcal{R}_i^+) - r_\theta(\mathcal{Q}_i, \mathcal{R}_i^-)))], \quad (6)$$

where  $r_\theta(\cdot)$  denotes the reward model,  $\sigma(\cdot)$  represents the sigmoid function, and  $\mathcal{R}_i^+/\mathcal{R}_i^-$  indicate true/false responses respectively. For our profiling task, we maintain a consistent query  $\mathcal{Q}_{inf.}$  while carefully constructing both positive responses  $\mathcal{R}^+$  and negative responses  $\mathcal{R}^-$ . We obtain  $\mathcal{R}^+$  through ChatGPT and create  $\mathcal{R}^-$  through two strategies:

- **Diverse Negative Sampling:** Multiple prompt templates generate varied low-quality responses, enabling the reward model to identify suboptimal outputs post instruction-tuning.
- **Profile Substitution:** Strategic replacement with similar users’ profiles helps the model discern subtle differences and prioritize personalized characteristics.

**Proximal Policy Optimization.** In our reinforcement learning framework, we treat the LLM  $\mathcal{M}$  as the policy to be optimized, with the reward model approximating the true reward function. The core optimization objective is formulated as:

$$\operatorname{argmax}_{\mathcal{M}} \mathbb{E}_{x_i \sim \mathcal{D}, y_i \sim \mathcal{M}} [R(y_i | x_i)]. \quad (7)$$

To optimize  $\mathcal{M}$  iteratively, we sample queries  $\mathcal{Q}_i$  from the dataset  $\mathcal{D}$  and generate corresponding responses  $\mathcal{R}_i$  using  $\mathcal{M}$ . We employ the Proximal Policy Optimization (PPO) algorithm and its

associated loss function for optimization. Following Schulman et al. (2017), we enhance the reward function with a KL divergence penalty term between the original LLM  $\mathcal{M}_0$  and the optimized LLM  $\mathcal{M}_\theta$ . This constraint effectively mitigates reward hacking—a phenomenon where models achieve high reward scores but poor human evaluation results. The final reward function  $R(\cdot)$  for a given query-response pair  $(\mathcal{Q}_i, \mathcal{R}_i)$  is defined as:

$$R(\mathcal{R}_i|\mathcal{Q}_i) = \hat{r}(\mathcal{R}_i|\mathcal{Q}_i) - \beta D_{KL}(\mathcal{M}_\theta(\mathcal{Q}_i)||\mathcal{M}_0(\mathcal{Q}_i)) \quad (8)$$

The comprehensive details of our instruction design across all fine-tuning stages, together with our methodology for constructing positive and negative training samples for the reinforcement learning-based reward model, are thoroughly documented in Appendix 6.6.

### 3 EVALUATION

In this section, we verify the effectiveness of RecLM by answering the following several questions:

- **RQ1:** How does our proposed RecLM enhance the performance of existing recommender systems, particularly in item cold-start scenarios?
- **RQ2:** What contributions do the instruction-tuning techniques and reinforcement learning enhancements make to overall recommendation performance?
- **RQ3:** How effective is our LLM-empowered user/item profiling system as an embedding function?
- **RQ4:** How does our method perform in terms of efficiency?
- **RQ5:** What advantages does our method have compared to existing LLM-enhanced recommenders?
- **RQ6:** How does the reinforcement learning-based profile generation refinement enhance the performance of our LLM-empowered profiling system?

#### 3.1 EXPERIMENTAL SETTINGS

To evaluate the effectiveness of our proposed method, we conduct extensive experiments using two public datasets: **MIND**<sup>1</sup> (Wu et al., 2020) and **Netflix**<sup>2</sup>, along with a large-scale dataset derived from real-world industrial data (referred to as Industrial for anonymity).

We assess the accuracy of the top- $K$  recommendation results using two widely adopted metrics: Recall@ $K$  (R@ $K$ ) and NDCG (N@ $K$ ), with  $K$  set to 20 by default. To reduce bias, we employ an all-rank evaluation strategy, where positive items in the test set are ranked alongside all non-interacted items for each user. The final metric is reported as the average score across all users in the test set.

We evaluate the effectiveness of our RecLM approach by integrating it with state-of-the-art recommender systems, allowing us to assess performance improvements in a model-agnostic manner compared to baseline models. The selected CF recommenders include non-graph methods such as BiasMF (Koren et al., 2009) and NCF (He et al., 2017), the GNN-enhanced method LightGCN (He et al., 2020), and graph contrastive learning approaches SGL (Wu et al., 2021) and SimGCL (Yu et al., 2022). Details regarding the datasets and baseline methods are provided in Appendices 6.1 and 6.2.

#### 3.2 PERFORMANCE COMPARISON (RQ1)

To demonstrate the effectiveness of our RecLM in enhancing performance, particularly in cold-start scenarios, we apply it to five common collaborative filtering methods. The "full-shot" setting corresponds to the complete dataset, while the "zero-shot" setting refers to the pure cold-start condition. The **Base** variant applies the cold-start recommendation paradigm to the baseline recommenders without any profiling enhancement via LLMs, whereas the **Augment** variant integrates RecLM into the base recommenders. Detailed settings and implementation information are provided in Appendices 6.3 and 6.5. The evaluation results in Table 1 reveal several interesting observations.

**(i) Performance Improvement in Integrated Recommenders.** We consistently find that integrating RecLM with backbone recommenders leads to enhanced performance compared to the base variant,

<sup>1</sup><https://msnews.github.io>

<sup>2</sup><https://www.kaggle.com/datasets/netflix-inc/netflix-prize-data>

Table 1: Performance comparison on MIND, Netflix and Industrial data in terms of *Recall* and *NDCG*. The superscript \* indicates the improvement is statistically significant where the p-value < 0.05.

| Dataset           |          | MIND           |                |                |                | Netflix        |                |                |                | Industrial     |                |                |                 |
|-------------------|----------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|-----------------|
| Backbone          | Variants | R@20           | R@40           | N@20           | N@40           | R@20           | R@40           | N@20           | N@40           | R@20           | R@40           | N@20           | N@40            |
| Full-Shot Setting |          |                |                |                |                |                |                |                |                |                |                |                |                 |
| BiasMF            | Base     | 0.0683         | 0.1039         | <b>0.0311</b>  | 0.0399         | 0.0449         | 0.0790         | 0.1451         | 0.1375         | 0.0078         | 0.0143         | 0.0046         | 0.0066          |
|                   | Augment. | <b>0.0719*</b> | <b>0.1353*</b> | 0.0272         | <b>0.0411*</b> | <b>0.0531*</b> | <b>0.0868*</b> | <b>0.1761*</b> | <b>0.1630*</b> | <b>0.0121*</b> | <b>0.0198*</b> | <b>0.0074*</b> | <b>0.0097*</b>  |
|                   | Improve. | 5.27% ↑        | 30.22% ↑       | 12.54% ↓       | 3.01% ↑        | 18.26% ↑       | 9.87% ↑        | 21.36% ↑       | 18.55% ↑       | 55.13% ↑       | 38.46% ↑       | 60.87% ↑       | 46.97% ↑        |
| NCF               | Base     | 0.0713         | 0.0985         | <b>0.0325</b>  | <b>0.0445</b>  | 0.0581         | 0.0936         | 0.1848         | 0.1721         | 0.0102         | 0.0076         | 0.0188         | 0.0091          |
|                   | Augment. | <b>0.0760*</b> | <b>0.1233*</b> | 0.0288         | 0.0414         | <b>0.0591*</b> | <b>0.0968*</b> | <b>0.1903*</b> | <b>0.1785*</b> | <b>0.0133*</b> | <b>0.0087*</b> | <b>0.0206*</b> | <b>0.0108**</b> |
|                   | Improve. | 6.59% ↑        | 25.18% ↑       | 11.38% ↓       | 6.97% ↓        | 1.72% ↑        | 3.42% ↑        | 2.98% ↑        | 3.72% ↑        | 30.39% ↑       | 14.47% ↑       | 9.57% ↑        | 18.68% ↑        |
| LightGCN          | Base     | 0.0389         | 0.0702         | 0.0150         | 0.0219         | 0.0467         | 0.0815         | 0.1488         | 0.1424         | 0.0096         | 0.0162         | 0.0059         | 0.0076          |
|                   | Augment. | <b>0.0788*</b> | <b>0.0983*</b> | <b>0.0337*</b> | <b>0.0384*</b> | <b>0.0652*</b> | <b>0.1026*</b> | <b>0.1703*</b> | <b>0.1606*</b> | <b>0.0143*</b> | <b>0.0225*</b> | <b>0.0087*</b> | <b>0.0107*</b>  |
|                   | Improve. | 102.57% ↑      | 40.03% ↑       | 124.67% ↑      | 75.34% ↑       | 39.61% ↑       | 25.89% ↑       | 14.45% ↑       | 12.78% ↑       | 48.96% ↑       | 38.89% ↑       | 47.46% ↑       | 40.79% ↑        |
| SGL               | Base     | 0.0345         | 0.0708         | 0.0127         | 0.0210         | 0.0277         | 0.0416         | 0.0855         | 0.0762         | 0.0078         | 0.0138         | 0.0050         | 0.0068          |
|                   | Augment. | <b>0.0732*</b> | <b>0.0967*</b> | <b>0.0367*</b> | <b>0.0421*</b> | <b>0.0788*</b> | <b>0.1204*</b> | <b>0.1958*</b> | <b>0.1831*</b> | <b>0.0133*</b> | <b>0.0221*</b> | <b>0.0080*</b> | <b>0.0106*</b>  |
|                   | Improve. | 112.17% ↑      | 36.58% ↑       | 188.98% ↑      | 100.48% ↑      | 184.48% ↑      | 189.42% ↑      | 129.01% ↑      | 140.29% ↑      | 70.51% ↑       | 60.14% ↑       | 60% ↑          | 55.88% ↑        |
| SimGCL            | Base     | 0.0421         | 0.0636         | 0.0155         | 0.0212         | 0.0231         | 0.0441         | 0.0810         | 0.0825         | 0.0042         | 0.0078         | 0.0026         | 0.0037          |
|                   | Augment. | <b>0.0576*</b> | <b>0.0908*</b> | <b>0.0232*</b> | <b>0.0329*</b> | <b>0.0567*</b> | <b>0.0908*</b> | <b>0.1782*</b> | <b>0.1673*</b> | <b>0.0128*</b> | <b>0.0205*</b> | <b>0.0080*</b> | <b>0.0099*</b>  |
|                   | Improve. | 36.82% ↑       | 42.77% ↑       | 49.68% ↑       | 55.19% ↑       | 145.45% ↑      | 105.90% ↑      | 120.00% ↑      | 102.79% ↑      | 204.76% ↑      | 162.82% ↑      | 207.69% ↑      | 167.57% ↑       |
| Zero-Shot Setting |          |                |                |                |                |                |                |                |                |                |                |                |                 |
| BiasMF            | Base     | 0.0096         | 0.0165         | 0.0031         | 0.0041         | 0.0311         | 0.0769         | 0.0167         | 0.0292         | 0.0038         | 0.0068         | 0.0020         | 0.0029          |
|                   | Augment. | <b>0.0246*</b> | <b>0.0373*</b> | <b>0.0107*</b> | <b>0.0135*</b> | <b>0.1381*</b> | <b>0.1490*</b> | <b>0.0828*</b> | <b>0.0584*</b> | <b>0.0056*</b> | <b>0.0103*</b> | <b>0.0026*</b> | <b>0.0040*</b>  |
|                   | Improve. | 156.25% ↑      | 126.06% ↑      | 245.16% ↑      | 229.27% ↑      | 344.05% ↑      | 93.76% ↑       | 395.81% ↑      | 100.00% ↑      | 47.37% ↑       | 51.47% ↑       | 30.00% ↑       | 37.93% ↑        |
| NCF               | Base     | 0.0301         | 0.0383         | 0.0080         | 0.0097         | 0.0480         | 0.1158         | 0.0196         | 0.0384         | 0.0044         | 0.0022         | 0.0056         | 0.0026          |
|                   | Augment. | <b>0.0424*</b> | <b>0.0469*</b> | <b>0.0112*</b> | <b>0.0122*</b> | <b>0.1700*</b> | <b>0.1774*</b> | <b>0.0984*</b> | <b>0.0974*</b> | <b>0.0051*</b> | <b>0.0031*</b> | <b>0.0088*</b> | <b>0.0041*</b>  |
|                   | Improve. | 40.86% ↑       | 22.45% ↑       | 40.00% ↑       | 25.77% ↑       | 254.17% ↑      | 53.20% ↑       | 402.04% ↑      | 153.65% ↑      | 15.91% ↑       | 40.91% ↑       | 57.14% ↑       | 57.69% ↑        |
| LightGCN          | Base     | 0.0138         | 0.0292         | 0.0046         | 0.0078         | 0.0974         | 0.1256         | 0.0446         | 0.0415         | 0.0092         | 0.0160         | 0.0051         | 0.0070          |
|                   | Augment. | <b>0.0196*</b> | <b>0.0389*</b> | <b>0.0064*</b> | <b>0.0086*</b> | <b>0.1371*</b> | <b>0.1453*</b> | <b>0.0697*</b> | <b>0.0459*</b> | <b>0.0133*</b> | <b>0.0188*</b> | <b>0.0090*</b> | <b>0.0106*</b>  |
|                   | Improve. | 42.03% ↑       | 33.22% ↑       | 39.13% ↑       | 10.26% ↑       | 40.76% ↑       | 15.68% ↑       | 56.28% ↑       | 10.60% ↑       | 44.57% ↑       | 17.50% ↑       | 76.47% ↑       | 51.43% ↑        |
| SGL               | Base     | 0.0162         | 0.0264         | 0.0062         | 0.0074         | 0.0385         | 0.1441         | 0.0274         | 0.0579         | 0.0065         | 0.0114         | 0.0036         | 0.0050          |
|                   | Augment. | <b>0.0254*</b> | <b>0.0450*</b> | <b>0.0089*</b> | <b>0.0107*</b> | <b>0.1126*</b> | <b>0.1756*</b> | <b>0.0384*</b> | <b>0.1066*</b> | <b>0.0111*</b> | <b>0.0176*</b> | <b>0.0066*</b> | <b>0.0084*</b>  |
|                   | Improve. | 56.79% ↑       | 70.45% ↑       | 43.55% ↑       | 44.59% ↑       | 92.47% ↑       | 21.86% ↑       | 40.15% ↑       | 84.11% ↑       | 70.77% ↑       | 54.39% ↑       | 83.33% ↑       | 68.00% ↑        |
| SimGCL            | Base     | 0.0164         | 0.0300         | 0.0055         | 0.0084         | 0.0793         | 0.1259         | 0.0336         | 0.0460         | 0.0078         | <b>0.0140</b>  | 0.0042         | 0.0059          |
|                   | Augment. | <b>0.0312*</b> | <b>0.0388*</b> | <b>0.0098*</b> | <b>0.0115*</b> | <b>0.1508*</b> | <b>0.1895*</b> | <b>0.1550*</b> | <b>0.1647*</b> | <b>0.0084*</b> | 0.0137         | <b>0.0044*</b> | 0.0059          |
|                   | Improve. | 90.24% ↑       | 29.33% ↑       | 78.18% ↑       | 36.90% ↑       | 90.16% ↑       | 50.52% ↑       | 361.31% ↑      | 258.04% ↑      | 7.69% ↑        | 2.14% ↓        | 4.76% ↑        | —               |

which relies on raw external item features and ID-based user embeddings in both **supervised** and **zero-shot** settings. This provides compelling evidence for the effectiveness of RecLM. We attribute these improvements to two key factors: *First*, for supervised recommendation scenarios, RecLM leverages instruction-tuned LLMs to generate accurate user and item profiles as auxiliary information, effectively enhancing the semantic representation of user preference. *Second*, our tuning paradigm guides the LLMs in capturing user collaborative relationships, allowing for the generation of high-quality, personalized profiles that demonstrate strong generalization in zero-shot scenarios.

**(ii) Outstanding Performance in Cold-Start Scenarios.** This improvement arises from our innovative modifications to the ID-embedding paradigm employed in current recommenders. By incorporating external features specifically designed to address the challenges of interaction data scarcity, we have significantly enhanced the effectiveness of these systems. Remarkably, we observe substantial performance improvements even in the relatively sparser MIND and Industrial datasets, where data limitations traditionally pose significant hurdles. By leveraging our RecLM for user and item profiling, we significantly enhance the generalization capabilities of existing recommenders.

**(iii) Practicality and Scalability for Real-World Deployment.** The results from the Industrial dataset demonstrate that RecLM consistently enhances the performance of recommenders in large-scale, highly sparse real-world scenarios. Furthermore, our user and item profile generation methods can be efficiently executed as an offline profiling system to support online applications, making them highly practical for real-world recommendations. To facilitate online recommendation systems, user and item profiles can be updated at regular intervals, such as daily or weekly. The performance improvements observed across various backbone models indicate that RecLM can easily adapt to a range of business models, significantly enhancing their overall effectiveness.

### 3.3 ABLATION STUDY (RQ2)

We conducted extensive experiments to validate the effectiveness of our proposed instruction tuning techniques by customizing three variants of RecLM: *GPT\_KD*, *Naive*, and *Mask*. Detailed descriptions of these variants can be found in Appendix 6.4. The results of our experiments are illustrated in Figure 2, allowing us to draw the following conclusions:

**(i) Advantage of Collaborative Instruction Tuning.** The results in Figure 2 show that using instruction tuning to capture collaborative relationships among users and items, along with the

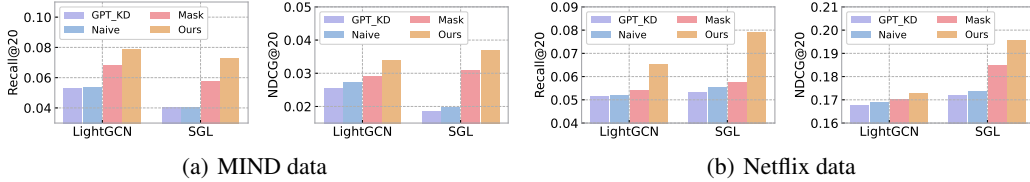


Figure 2: Ablation study on the LLM tuning techniques in the RecLM framework.

masking tuning strategy (*Mask*), significantly enhances performance compared to *GPT\_KD*. This improvement suggests that our tuning solution generates more precise, high-quality profiles by leveraging collaborative information effectively. In contrast, profiling based solely on user interaction history has limitations, as it lacks the guidance from collaborative insights. Consequently, this approach often results in less accurate profiles that may include noisy interaction records.

**(ii) Effectiveness of the Masking-Based Tuning Strategy.** Although the *Naive* variant also employs a two-round dialogue-based instruction tuning technique similar to the *Mask* variant, its improvement over the *GPT\_KD* variant is limited. This underscores the advantages of the masking-based tuning strategy, which effectively utilizes responses from the two-round dialogue to update the weights of the LLM and guide its learning of collaborative relationships between users.

**(iii) Benefits of Reinforcement Learning-Based Profile Generation Refinement.** The results indicate that the *Mask* variant performs significantly worse than RecLM. This finding suggests that the proposed reinforcement learning (RL)-based profile generation refinement technique effectively addresses the noise issues and over-smoothing problems associated with the collaborative instruction-tuning paradigm. As a result, it enables the LLM to generate more accurate and personalized profiles.

### 3.4 EFFECTIVENESS OF LLM-EMPOWERED PROFILING SYSTEM IN RECLM (RQ3)

To investigate the impact of our LLM-empowered profiling system on user and item feature enhancements, we developed two variants of RecLM: one that excludes user feature enhancement (*i.e.*, w/o User Aug.) and another that excludes item feature enhancement (*i.e.*, w/o Item Aug.). The experiments were conducted on the MIND and Netflix datasets using the full-shot setting, with LightGCN and SGL as the backbone models. The evaluation results are presented in Table 2, allowing us to draw the following conclusions.

Table 2: Performance *w.r.t.* various aug. variants.

| Dataset  |               | MIND          |               | Netflix       |               |
|----------|---------------|---------------|---------------|---------------|---------------|
| Backbone | Variants      | R@20          | N@20          | R@20          | N@20          |
| LightGCN | Base          | 0.0389        | 0.0150        | 0.0467        | 0.1488        |
|          | w/o User Aug. | 0.0302        | 0.0123        | 0.0384        | 0.1213        |
|          | w/o Item Aug. | 0.0719        | 0.0287        | 0.0505        | 0.1621        |
|          | RecLM         | <b>0.0788</b> | <b>0.0337</b> | <b>0.0652</b> | <b>0.1703</b> |
| SGL      | Base          | 0.0345        | 0.0127        | 0.0277        | 0.0855        |
|          | w/o User Aug. | 0.0253        | 0.0093        | 0.0173        | 0.0578        |
|          | w/o Item Aug. | 0.0719        | 0.0289        | 0.0502        | 0.1546        |
|          | RecLM         | <b>0.0732</b> | <b>0.0367</b> | <b>0.0788</b> | <b>0.1958</b> |

**(i) User-Side Feature Enhancement.** The exclusion of user-side feature enhancements (*i.e.*, w/o User Aug.) results in a significant decline in performance across both evaluated datasets and backbone models. This underscores the critical role of our RecLM as the profiling system for improving performance. Relying solely on the original ID embedding for the user side is insufficient for effectively capturing and modeling user preferences. We attribute this outcome to both the effective extraction of text features and the successful integration of graph and textual information.

**(ii) Item-Side Feature Enhancement.** The exclusion of item-side feature enhancements (*i.e.*, w/o Item Aug.) also leads to a noticeable decline in the recommender’s performance. Interestingly, when item-side feature enhancements are retained without incorporating any user-side feature enhancements (*i.e.*, w/o User Aug.), the performance can drop even below that of the Base variant. This discrepancy can be attributed to the interplay between raw and enhanced features on the item side, which creates a complex dynamic. Relying solely on ID embedding for the user side proves inadequate for effectively modeling user preferences.

### 3.5 TRAINING EFFICIENCY ANALYSIS OF RECLM (RQ4)



To evaluate the efficiency of our RecLM approach, we conduct both a theoretical complexity analysis and an empirical running time test. **Theoretical Analysis:** The time complexity of the MLP used to transfer textual features  $\mathbf{f} \in \mathbb{R}^{d_t}$  of items into the model’s latent space  $\mathbb{R}^d$  is  $\mathcal{O}(N \times (d_t \times d + d \times d))$ , where  $N$  represents the number of nodes, and  $d_t$  and  $d$  denote the dimensionalities of the original text features and the latent space, respectively. **Empirical Evaluation:** We present the per-epoch training time in Table 3. The evaluation was conducted on a server equipped with NVIDIA A100 GPUs (40 GB memory). The results indicate that for larger models (e.g., GNN-based methods), our RecLM requires relatively little additional time, often falling below 10%. In denser datasets like Netflix, this additional time can be reduced to under 5%. Even for smaller recommenders, the maximum additional time is approximately 25%. Given the substantial improvements in recommendation provided by our method, the incurred costs are considered acceptable.

Table 3: Training efficiency *w.r.t.* integration with various recommenders.

| Dataset    | Recommender | Base   | RecLM  | Cost    |
|------------|-------------|--------|--------|---------|
| MIND       | BiasMF      | 0.72s  | 0.85s  | +18.06% |
|            | NCF         | 0.76s  | 0.85s  | +11.84% |
|            | LightGCN    | 0.79s  | 0.86s  | +8.86%  |
|            | SGL         | 1.93s  | 2.01s  | +4.15%  |
|            | SimGCL      | 2.63s  | 2.69s  | +2.28%  |
| Netflix    | BiasMF      | 14.38s | 16.42s | +14.19% |
|            | NCF         | 15.02s | 17.17s | +14.31% |
|            | LightGCN    | 20.47s | 20.95s | +2.34%  |
|            | SGL         | 64.98s | 65.08s | +0.15%  |
|            | SimGCL      | 44.02s | 44.61s | +1.34%  |
| Industrial | BiasMF      | 7.07s  | 8.85s  | +25.18% |
|            | NCF         | 7.58s  | 8.45s  | +11.48% |
|            | LightGCN    | 9.33s  | 10.25s | +9.86%  |
|            | SGL         | 32.34s | 32.87s | +1.64%  |
|            | SimGCL      | 85.41s | 86.52s | +1.30%  |

### 3.6 COMPARISON WITH EXISTING LLM-ENHANCED METHODS(RQ5)

We further compare RecLM with the existing work LLMRec (Wei et al., 2024), which also enhances recommendation systems using LLMs, to highlight the superiority of our proposed instruction-tuning technique. The experimental results are presented in Figure 3. Specifically, LLMRec generates profiles for items and users by directly calling the LLM’s API without fine-tuning for the profile generation task. This approach fails to effectively leverage the collaborative relationships among users. As a result, RecLM demonstrates significant performance advantages across two public datasets, leading to notable improvements in the performance of the base models.

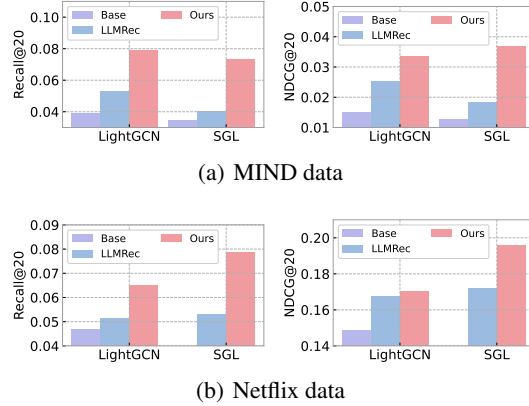


Figure 3: Performance Comparison with LLMRec.

### 3.7 CASE STUDY(RQ6)

To intuitively explore the contribution of reinforcement learning to the personalization of generated profiles, we conducted a case study using the MIND dataset. In this study, as shown in Figure 4, the target user for whom the profile is being generated is *User 49*. This user has interacted with two items: *Item 472* and *Item 1572*. Additionally, we identified three similar users who provide collaborative information: *User 11451*, *User 20522*, and *User 341*.

The user profile generated for *User 49* after instruction tuning, but without reinforcement learning (RL) tuning, contains several irrelevant keywords related to the interacted items, such as "Foodie," "Food and drink," and "Horoscope." Notably, these terms also appear in the profiles of *User 11451* and *User 20522*, suggesting that the generated profile is overly influenced by too many collaborative users. In contrast, the profile generated for *User 49* after RL tuning effectively preserves the preferences indicated in the interaction history while incorporating relevant implicit keywords from collaborative users. For example, the term "pop culture" is derived from *User 341*'s profile. This approach provides precise and valuable

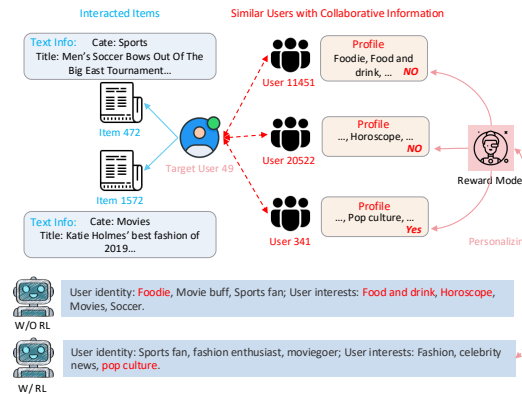


Figure 4: Generated profiles w/ and w/o RL.

---

additional information for modeling *User 49*'s preferences. We attribute this improvement to our proposed RL-based personalized feature enhancement techniques, which effectively address the noise and over-smoothing issues that can arise during the instruction-tuning process.

## 4 RELATED WORK

### 4.1 ID-BASED RECOMMENDER SYSTEMS

In recommender systems, numerous collaborative filtering models have been proposed to map users and items into latent representations based on user/item IDs (Koren et al., 2021; Su & Khoshgoftaar, 2009). These methods have evolved significantly, starting from early matrix factorization techniques, such as BiasMF (Koren et al., 2009), to the introduction of Neural Collaborative Filtering (NCF) with the advent of neural networks (He et al., 2017). Recently, advancements in Graph Neural Networks (GNNs) have opened promising avenues for constructing bipartite graphs based on user-item interaction history, allowing for the capture of high-order collaborative relationships. GNN-based methods, including NGCF (Wang et al., 2019), GCCF (Chen et al.), and LightGCN (He et al., 2020), have demonstrated state-of-the-art performance, enhancing the effectiveness of recommendation.

Additionally, researchers have incorporated self-supervised learning (SSL) techniques as supplementary learning objectives to improve the robustness of recommenders and address challenges related to data sparsity and noise (Yu et al., 2023). Contrastive learning (CL), a widely adopted SSL technique, has been effectively applied in CF research through approaches such as SGL (Wu et al., 2021), SimGCL (Yu et al., 2022), NCL (Lin et al., 2022), and AdaGCL (Jiang et al., 2023). Despite these advancements, ID-based recommenders still face significant limitations, particularly in completely cold-start scenarios and in terms of model transferability (Yuan et al., 2023).

### 4.2 LARGE LANGUAGE MODELS (LLMs) FOR RECOMMENDATION

The application of large language models (LLMs) in recommender systems has garnered significant attention (Fan et al., 2024; Lin et al., 2023; Liu et al., 2023). Current approaches can be categorized into two main types. The first category includes methods such as P5 (Geng et al., 2022) and ChatREC (Gao et al., 2023), which emphasize designing prompts aligned with recommendation tasks, utilizing the LLM directly as the inference model. The second category enhances existing recommenders by integrating LLMs while still relying on traditional collaborative filtering methods (Wang et al., 2024). For instance, LLMRec (Wei et al., 2024) strengthens the user-item interaction graph through LLM-based graph augmentation, while RLMRec (Ren et al., 2024) combines LLM-enhanced text embeddings with GNN-based user/item representations. However, these approaches often lack fine-tuning tailored to specific recommendation tasks, primarily focusing on full-shot scenarios.

In contrast, our work introduces a novel instruction-tuning technique for an open-source LLM, allowing it to adapt to specific recommendation tasks and effectively capture collaborative information for profile generation. While methods like InstructRec (Zhang et al., 2023) and TALLRec (Bao et al., 2023) align LLM capabilities with recommendation tasks, they struggle with scalability due to instruction-question-answering prompts and exhibit poor generalization on sparse data. Our approach enhances the generalization ability of existing recommender systems in the face of data scarcity and noise, while maintaining efficiency in handling large-scale data in practical scenarios.

## 5 CONCLUSION

In this work, by seamlessly integrating large language models with collaborative filtering, we have introduced a novel instruction-tuning paradigm that equips language models with the critical ability to capture complex user-item interactions and behavioral preferences. This model-agnostic approach can be easily plugged into existing recommender systems, dramatically enhancing their generalization capacity, particularly in data-scarce scenarios where traditional ID-based methods often struggle. The two key innovations of RecLM - the fusion of external features with collaborative patterns through specialized instruction tuning, and the reinforcement learning-based personalized feature enhancement framework - address the fundamental challenges of cold-start profiling and data noise inherent in recommendation tasks. Comprehensive evaluations demonstrate the significant advantages and practical compatibility of our approach with state-of-the-art recommender systems.

---

## REFERENCES

- Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In *RecSys*, pp. 1007–1014, 2023.
- Lei Chen, Le Wu, Richang Hong, Kun Zhang, and Meng Wang. Revisiting graph based collaborative filtering: A linear residual graph convolutional network approach. In *AAAI’20*.
- Wenqi Fan, Zihuai Zhao, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Jiliang Tang, and Qing Li. Recommender systems in the era of large language models (llms). *TKDE*, 2024.
- Yunfan Gao, Tao Sheng, Youlin Xiang, Yun Xiong, Haofen Wang, and Jiawei Zhang. Chatrec: Towards interactive and explainable llms-augmented recommender system. *arXiv preprint arXiv:2303.14524*, 2023.
- Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *RecSys*, pp. 299–315, 2022.
- Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering. In *WWW*, pp. 173–182, 2017.
- Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *SIGIR*, pp. 639–648, 2020.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- Mohsen Jamali and Martin Ester. A matrix factorization technique with trust propagation for recommendation in social networks. In *RecSys*, pp. 135–142, 2010.
- Yangqin Jiang, Chao Huang, and Lianghao Xia. Adaptive graph contrastive learning for recommendation. In *KDD*, pp. 4252–4261, 2023.
- Yangqin Jiang, Lianghao Xia, Wei Wei, Da Luo, Kangyi Lin, and Chao Huang. Diffmm: Multi-modal diffusion model for recommendation. In *ACM MM*, pp. 7591–7599, 2024.
- Yehuda Koren, Robert Bell, et al. Matrix factorization techniques for recommender systems. *Computer*, (8):30–37, 2009.
- Yehuda Koren, Steffen Rendle, and Robert Bell. Advances in collaborative filtering. *Recommender systems handbook*, pp. 91–142, 2021.
- Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Hao Zhang, Yong Liu, Chuhan Wu, Xiangyang Li, Chenxu Zhu, et al. How can recommender systems benefit from large language models: A survey. *TOIS*, 2023.
- Zihan Lin, Changxin Tian, Yupeng Hou, and Wayne Xin Zhao. Improving graph collaborative filtering with neighborhood-enriched contrastive learning. In *WWW*, pp. 2320–2329, 2022.
- Peng Liu, Lemei Zhang, and Jon Atle Gulla. Pre-train, prompt, and recommendation: A comprehensive survey of language modeling paradigm adaptations in recommender systems. *TACL*, 11: 1553–1571, 2023.
- Xubin Ren, Wei Wei, Lianghao Xia, Lixin Su, Suqi Cheng, Junfeng Wang, Dawei Yin, and Chao Huang. Representation learning with large language models for recommendation. In *WWW*, 2024.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

- 
- Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. Learning to summarize with human feedback. *NeurIPS*, pp. 3008–3021, 2020.
- Xiaoyuan Su and Taghi M Khoshgoftaar. A survey of collaborative filtering techniques. *Advances in artificial intelligence*, 2009, 2009.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- Jianling Wang, Raphael Louca, Diane Hu, Caitlin Cellier, James Caverlee, and Liangjie Hong. Time to shop for valentine’s day: Shopping occasions and sequential recommendation in e-commerce. In *WSDM*, pp. 645–653, 2020.
- Jianling Wang, Haokai Lu, James Caverlee, Ed H Chi, and Minmin Chen. Large language models as data augmenters for cold-start item recommendation. In *WWW*, pp. 726–729, 2024.
- Xiang Wang, Xiangnan He, Meng Wang, et al. Neural graph collaborative filtering. In *SIGIR*, 2019.
- Wei Wei, Xubin Ren, Jiabin Tang, Qinyong Wang, Lixin Su, Suqi Cheng, Junfeng Wang, Dawei Yin, and Chao Huang. Llmrec: Large language models with graph augmentation for recommendation. In *WSDM*, pp. 806–815, 2024.
- Fangzhao Wu, Ying Qiao, Jiun-Hung Chen, Chuhan Wu, Tao Qi, Jianxun Lian, Danyang Liu, Xing Xie, Jianfeng Gao, Winnie Wu, et al. Mind: A large-scale dataset for news recommendation. In *ACL*, pp. 3597–3606, 2020.
- Jiancan Wu, Xiang Wang, Fuli Feng, Xiangnan He, Liang Chen, Jianxun Lian, and Xing Xie. Self-supervised graph learning for recommendation. In *SIGIR*, pp. 726–735, 2021.
- Liang Wu, Diane Hu, Liangjie Hong, and Huan Liu. Turning clicks into purchases: Revenue optimization for product search in e-commerce. In *SIGIR*, pp. 365–374, 2018.
- Tiansheng Yao, Xinyang Yi, Derek Zhiyuan Cheng, Felix Yu, Ting Chen, Aditya Menon, Lichan Hong, Ed H Chi, Steve Tjoa, Jieqi Kang, et al. Self-supervised learning for large-scale item recommendations. In *CIKM*, pp. 4321–4330, 2021.
- Junliang Yu, Hongzhi Yin, Xin Xia, Tong Chen, Lizhen Cui, and Quoc Viet Hung Nguyen. Are graph augmentations necessary? simple graph contrastive learning for recommendation. In *SIGIR*, pp. 1294–1303, 2022.
- Junliang Yu, Hongzhi Yin, Xin Xia, Tong Chen, Jundong Li, and Zi Huang. Self-supervised learning for recommender systems: A survey. *TKDE*, 2023.
- Zheng Yuan, Fajie Yuan, Yu Song, Youhua Li, Junchen Fu, Fei Yang, Yunzhu Pan, and Yongxin Ni. Where to go next for recommender systems? id-vs. modality-based recommender models revisited. In *SIGIR*, pp. 2639–2649, 2023.
- Ruohan Zhan, Changhua Pei, Qiang Su, Jianfeng Wen, Xueliang Wang, Guanyu Mu, Dong Zheng, Peng Jiang, and Kun Gai. Deconfounding duration bias in watch-time prediction for video recommendation. In *KDD*, pp. 4472–4481, 2022.
- Fanjin Zhang, Jie Tang, Xueyi Liu, Zhenyu Hou, Yuxiao Dong, Jing Zhang, Xiao Liu, Ruobing Xie, Kai Zhuang, Xu Zhang, et al. Understanding wechat user preferences and “wow” diffusion. *TKDE*, pp. 6033–6046, 2021.
- Junjie Zhang, Ruobing Xie, Yupeng Hou, Xin Zhao, Leyu Lin, and Ji-Rong Wen. Recommendation as instruction following: A large language model empowered recommendation approach. *TOIS*, 2023.

---

## 6 APPENDIX

### 6.1 DATASET DESCRIPTIONS

The statistical characteristics of the three datasets used in our experiments are summarized in Table 4. Below, we provide detailed descriptions of each dataset:

Table 4: Statistics of the experimental datasets.

| <i>Statistics</i>      | <b>MIND</b> | <b>Netflix</b> | <b>Industrial</b> |
|------------------------|-------------|----------------|-------------------|
| <b># User</b>          | 57128       | 16835          | 117433            |
| <b># Overlap. Item</b> | 1020        | 6232           | 72417             |
| <b># Snapshot</b>      | daily       | yearly         | daily             |
| Training Set           |             |                |                   |
| <b># Item</b>          | 2386        | 6532           | 152069            |
| <b># Interactions</b>  | 89734       | 1655395        | 858087            |
| <b># Sparsity</b>      | 99.934%     | 98.495%        | 99.995%           |
| Test Set               |             |                |                   |
| <b># Item</b>          | 2461        | 8413           | 158155            |
| <b># Interactions</b>  | 87974       | 1307051        | 876415            |
| <b># Sparsity</b>      | 99.937%     | 99.077%        | 99.995%           |

- **MIND Data:** A large-scale news recommendation benchmark dataset. We extracted data spanning two consecutive days, with one day allocated for training and the subsequent day for testing. Each news article contains rich textual features including category labels, titles, and abstract content.
- **Netflix Data:** This dataset is derived from the Netflix Prize Data competition on Kaggle, comprising implicit user-movie interactions. For our experiments, we extracted viewing histories spanning two consecutive years (2005-2006), with the first year’s data allocated for training and the subsequent year for testing. The textual features for each movie item are represented by its corresponding title.
- **Industrial Data:** A large-scale proprietary dataset collected from a major online content platform (anonymized for privacy). This comprehensive news article collection serves millions of daily active users and generates substantial user-article interactions. For our evaluation, we extracted interaction data from two consecutive days (Day N and Day N+1), utilizing them as training and testing sets respectively. Each article is represented by its title as the primary textual feature.

### 6.2 BASE MODEL DESCRIPTIONS

We provide detailed descriptions of the baseline methods employed in our comparative evaluation:

- **BiasMF** (Koren et al., 2009): A matrix factorization-based approach that enhances recommendation accuracy by incorporating user and item bias vectors to capture individual preference patterns.
- **NCF** (He et al., 2017): A neural collaborative filtering framework that replaces traditional matrix factorization’s dot-product operation with multi-layer neural networks, enabling the capture of complex user-item interactions. In our implementation, we specifically utilize the NeuMF variant.
- **LightGCN** (He et al., 2020): A graph-based recommendation model that leverages user-item interaction graphs through a simplified layer-wise propagation scheme, employing only linear transformations and element-wise operations to aggregate neighborhood information.
- **SGL** (Wu et al., 2021): An enhanced version of LightGCN that incorporates self-supervised contrastive learning. It employs various data augmentation techniques, including random walks and node/edge dropout, to generate diverse graph views for contrastive learning.
- **SimGCL** (Yu et al., 2022): A streamlined contrastive learning framework that generates contrastive views by directly injecting uniform noise into the embedding space, eliminating the need for explicit graph augmentation operations.

### 6.3 EXPERIMENTAL SETTINGS FOR PERFORMANCE COMPARISON

We conducted our performance evaluation (detailed in Sec. 3.2) under two distinct testing scenarios:

- **Full-Shot Setting:** Utilizes the complete test set, including items that appeared in the training phase, representing conventional recommendation scenarios.
- **Zero-Shot Setting:** Exclusively evaluates items absent from the training set, specifically designed to assess cold-start recommendation capability where items have no historical interaction data.

Our experimental framework compares two implementation variants:

- **Base Variant:** Implements our cold-start recommendation paradigm using only user ID embeddings and item text embeddings, without LLM-generated profiles.
- **Augmented Variant:** Fully integrates the proposed RecLM framework with traditional recommenders, incorporating LLM-generated item profiles.

This comparative setup enables systematic evaluation of how LLM-generated profiles enhance recommender performance, particularly in addressing cold-start challenges.

## 6.4 ABLATION STUDY CONFIGURATION

We evaluate four distinct variants of our model to assess the contribution of each component:

- **GPT\_KD Variant:** Implements basic large language model (LLM) fine-tuning using ChatGPT3.5-generated user profiles, as detailed in Sec. 2.3.1.
- **Naive Variant:** Builds upon GPT\_KD by incorporating instruction tuning, but restricts the dialogue to single-turn interactions, diverging from our proposed two-turn approach (Sec. 2.3.2).
- **Mask Variant:** Extends GPT\_KD with both two-turn instruction tuning and masking-based strategies, representing an intermediate step in our framework.
- **Ours (RecLM):** Represents our complete framework, enhancing the Mask variant with RL-based personalized feature optimization.

## 6.5 IMPLEMENTATION DETAILS

### 6.5.1 PARAMETER-EFFICIENT FINE-TUNING STRATEGY

To optimize LLM adaptation while preserving their core reasoning capabilities, we employ Parameter-Efficient Fine-Tuning (PEFT) methodology. Specifically, we utilize Low-Rank Adaptation (LoRA) (Hu et al., 2021) for fine-tuning Llama2-7b-chat (Touvron et al., 2023), enabling efficient task-specific adaptation while maintaining the model’s fundamental knowledge base.

### 6.5.2 BASE RECOMMENDER INTEGRATION AND HYPERPARAMETER CONFIGURATION

We systematically integrated RecLM into various base recommenders, conducting comprehensive hyperparameter optimization to ensure fair comparison. All models are implemented using PyTorch framework, utilizing Adam optimizer with default parameters and Xavier initialization. The training process employs a batch size of 4096, with embedding vectors dimensionality set to 32 and a learning rate of  $1e^{-3}$ . The  $\mathcal{L}_2$  regularization coefficient is optimized through a search space of  $\{1e^{-3}, 1e^{-4}, 1e^{-5}, 1e^{-6}, 1e^{-7}\}$ . For GNN-based models (LightGCN, SGL, and SimGCL), we set the number of GCN layers to 2. Additionally, for self-supervised learning (SSL)-based models (SGL and SimGCL), the temperature coefficient is tuned within the range of  $\{0.1, 0.5, 1.0\}$ .

## 6.6 INSTRUCTION DESIGNS

In this section, we provide a comprehensive overview of the instructions utilized for fine-tuning at each stage of our process. We will also discuss the methodologies employed to construct both positive and negative training samples for the reinforcement learning reward model.

- **Instruction Generation.** As illustrated in Figure 5, our instruction generation is conducted based on the knowledge distillation process through leveraging ChatGPT’s capabilities by feeding it two key inputs: user-specific textual information and interaction-related item descriptions. The LLMs

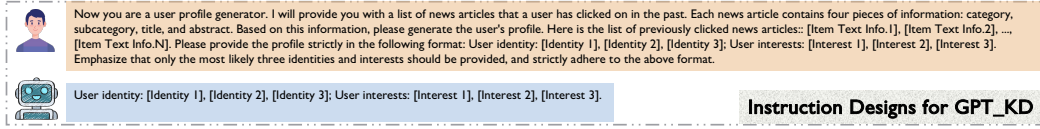


Figure 5: Instruction generation via ChatGPT knowledge distillation.

then synthesize this information to generate comprehensive user profiles that capture both personal identity characteristics and specific interest patterns.

- **Two-Turn Collaborative Instruction Design.** Our instruction tuning framework implements a carefully structured two-turn dialogue approach (Figure 6). **Initial Setup:** The system receives foundational instructions to establish LLMs’ understanding of collaborative filtering principles.

**First Turn:** In the first turn, we provide input instructions containing interaction histories from similar users and relevant item details. These similar users are identified through a graph collaborative filtering model, followed by embedding-based similarity calculations. The LLMs then process this information to generate comprehensive user profiles for each referenced user.

**Second Turn:** In the second turn, the LLMs perform prediction tasks by analyzing the previously generated user profiles in conjunction with specific item characteristics. Based on collaborative filtering principles, the model evaluates potential user-item interactions and outputs definitive binary predictions (Yes/No) regarding interaction likelihood.

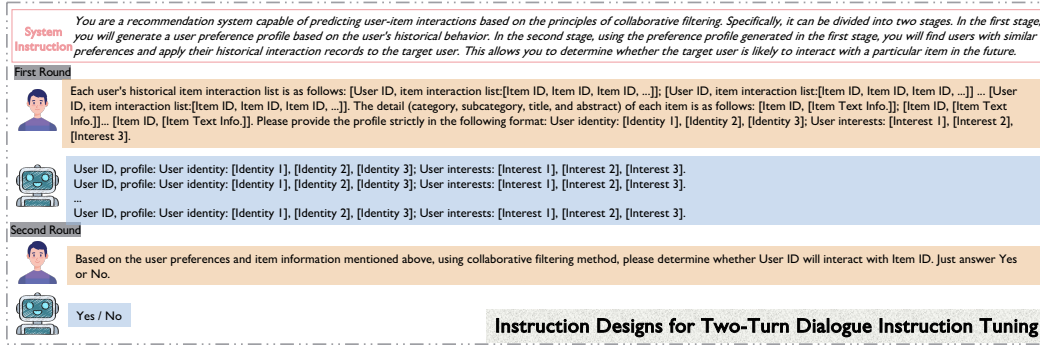


Figure 6: Instruction designs for two-turn instruction tuning.

- **User Profile Generation Instruction Design.** Following the instruction tuning, the LLMs develop proficiency in profile generation with collaborative awareness. As illustrated in Figure 7, we implement a systematic instruction framework for user profile generation that consists of three key components. The process begins with system-level instructions that reinforce the LLMs’ understanding of collaborative filtering principles. The input layer then incorporates two essential elements: (1) interaction histories from multiple similar users, including the target user requiring profile generation, and (2) comprehensive textual descriptions of all relevant items. Based on this information, the LLMs synthesize and output a detailed profile for the target user.

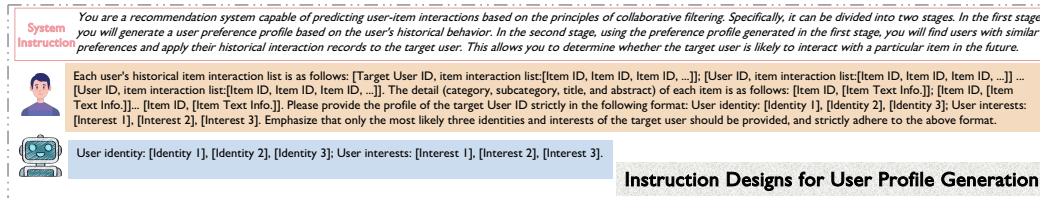


Figure 7: Instruction designs for user profile generation.

- **Item Profile Generation Instruction Design.** Following user profile generation, we develop item profiles to ensure semantic alignment between user and item features. The item profile represents

a target user’s profile specific to a particular item, and we implement a dual-phase approach for comprehensive coverage: In the first phase, we generate profiles for items with existing user interactions by leveraging the profiles of their interacting users. This establishes direct user-item semantic connections. For cold-start items, the second phase employs similarity-based matching using raw item embeddings, followed by LLM-based profile inference. As illustrated in Figure 8, the instruction framework processes inputs comprising: (1) a target item and its similar items, (2) detailed textual information for all items, and (3) existing profiles of similar items. The LLMs then synthesize this information to generate the target item’s profile, thereby enhancing the overall semantic coherence of the recommendation system.

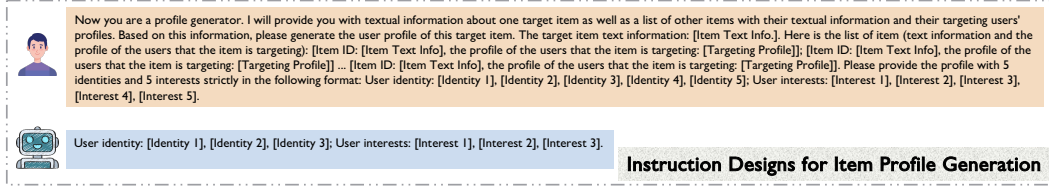


Figure 8: Instruction designs for item profile generation.

- **Reward Model Training: Response Construction.** The success of our personalized feature enhancement through reinforcement learning (Sec 2.4) depends critically on the quality of the reward model training data. This training data addresses two key challenges: instruction-tuning noise and collaborative feature over-smoothing. As illustrated in Figure 9, we construct positive samples through a combination of state-of-the-art LLMs (*e.g.*, ChatGPT) and manual validation. For negative samples, we establish a complementary sampling strategy:
  - **Similar User Profiles:** These samples train the reward model to identify subtle profile distinctions, thereby mitigating over-smoothing effects.
  - **Low-Quality Responses:** This category includes profiles with missing information or repetitive content, serving as explicit negative examples for model training.

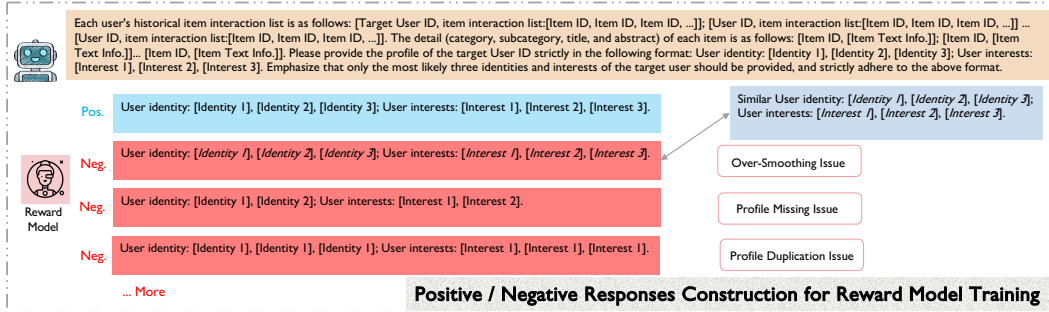


Figure 9: Positive/Negative responses construction for reward model training.

## 6.7 FUTURE WORK EXPLORATION

Future research directions for RecLM could explore two promising avenues: First, extending the framework to multi-modal contexts, where user-item interactions involve diverse data types such as images, videos, and audio. This would require developing modal-specific instruction tuning strategies and designing cross-modal alignment mechanisms to maintain semantic consistency across different modalities. Second, investigating the potential of smaller, more efficient language models for profile generation. While our current implementation leverages Llama2-7b, exploring distilled or compressed models could significantly reduce computational overhead while maintaining profile quality. This could involve techniques like knowledge distillation from larger to smaller models specifically trained for recommendation tasks, or developing specialized lightweight architectures that focus exclusively on user/item profiling capabilities.